BrewFlow Final Project

Teil 4 Automatisierung in Databricks

Projektdokumentation zur Migration, automatisierten Ausführung und fachlichen Validierung der ETL-Pipeline

Dokumentationsstand 7. Oktober 2026

Ziel und Ergebnis

Die in PostgreSQL entwickelte BrewFlow-Pipeline wurde nach Databricks übertragen. Im Katalog workspace bestehen die drei Ebenen brewflow_oltp, brewflow_staging und brewflow_warehouse. Drei gespeicherte SQL-Queries bilden den Ladeprozess und sind in einem Lakeflow-Job miteinander verbunden.

Der manuelle Job-Test war erfolgreich. Der tägliche Zeitplan ist aktiv auf 02:00 Uhr Europe/Berlin eingestellt. Laut Ergebnisbericht und Zusammenfassung wurde auch der automatisch gestartete Nachtlauf am 7. Oktober 2026 erfolgreich abgeschlossen. Die folgenden Abbildungen dokumentieren die Job-Konfiguration und den manuellen Testlauf.

Die Änderungsszenarien sind bestanden: Sofias Loyalty-Upgrade erzeugte eine historische Bronze- und eine aktuelle Silver-Version; Priya und ihre Bestellung wurden ergänzt; eine verspätete Bestellung aus Januar 2024 erhielt Sofias historischen Bronze-Schlüssel. Die Abschlusskontrolle bestätigt 34 eindeutige Fakten, 0 fehlende Positionen und 177,00 Gesamtumsatz.

Bereich	Bestätigter Stand
Quellbestand vor Simulation	10 Kunden, 5 Filialen, 8 Produkte, 22 Bestellungen, 32 Positionen
Warehouse nach Simulation	11 aktuelle Kunden, 12 Kundenversionen, 34 Fakten
Automatisierung	Drei abhängige SQL-Aufgaben; manueller Test erfolgreich, automatischer Nachtlauf laut Ergebnisbericht und Zusammenfassung erfolgreich
Power BI	Verbunden; fünf Beziehungen, Kundenhistorie, Kennzahlen und Filter geprüft

Umsetzung und Geltungsbereich

Identity-Schlüssel und die generierte Umsatzspalte werden beim Einfügen ausgelassen. Produkte und Filialen verwenden MERGE mit stabilen Surrogatschlüsseln. Die Faktentabelle ergänzt nur neue order_item_id-Werte. Die Wiederholbarkeit wurde auf unveränderten Daten geprüft.

Schritte 1 bis 3 Quelle und Architektur

Aufgabenstellung Die OLTP-Quelle intern in Databricks bereitstellen und die drei Schemaebenen entwerfen.

Schritt Die fünf OLTP-Tabellen als Delta-Tabellen im Katalog workspace anlegen, Beispieldaten aus Teil 1 laden und kontrollieren. Anschließend Genie Code für den Architekturentwurf verwenden.

Aktion und SQL PostgreSQL search_path wurde durch die Databricks-Katalog- und Schemaauswahl ersetzt. Die Quelltabellen verwenden STRING und enthalten keine REFERENCES-Klauseln.

```
-- Umgebung und Drei Schema Architektur
USE CATALOG workspace;
CREATE SCHEMA IF NOT EXISTS brewflow_oltp;
CREATE SCHEMA IF NOT EXISTS brewflow_staging;
CREATE SCHEMA IF NOT EXISTS brewflow_warehouse;
SHOW SCHEMAS IN workspace LIKE 'brewflow*';
```

Kontrolle Die initialen Zeilenanzahlen sind 10 / 5 / 8 / 22 / 32. Die zusammengefasste Bestellsummenprüfung bestätigt 22 geprüfte Bestellungen, 22 Übereinstimmungen und 0 Abweichungen.

Ergebnis Die interne Quelle und die drei Schemaebenen sind erfolgreich eingerichtet.

Abbildung 1 Initiale Zeilenanzahlen der OLTP Quelle

	tbl	rows
1	customers	10
2	stores	5
3	products	8
4	orders	22
5	order_items	32

Abbildung 2 Vollständige Bestellsummenprüfung

	checked_orders	matching_orders	mismatching_orders
1	22	22	0

Der Genie-Entwurf wurde als Markdown-Notiz festgehalten und fachlich geprüft. Übernommen wurden Staging, SCD Type 2, die Bestellposition als Faktengranularität und die Kalenderdimension. Spaltennamen wurden an Teil 2 angeglichen. Zusätzliche Staging-Metadaten wurden nicht eingeführt.

Schritt 4 Warehouse Struktur

Aufgabenstellung Fünf Dimensionen und eine Faktentabelle als Delta-Tabellen erstellen.

Schritt Genie Code mit den konkreten Spaltenvorgaben zur DDL-Erstellung nutzen, den Vorschlag prüfen und den DDL-Block ausführen.

Aktion und SQL Alle Tabellen sind vollständig qualifiziert. Identity-Schlüssel sind BIGINT; Dimensionsverweise sind logische Beziehungen ohne erzwungene Fremdschlüssel-Constraints.

```
-- Tatsächlich gespeicherte Definitionen kontrollieren
SHOW TABLES IN workspace.brewflow_warehouse;
SHOW CREATE TABLE workspace.brewflow_warehouse.dim_customer;
SHOW CREATE TABLE workspace.brewflow_warehouse.fact_order_items;

-- Bestätigte Umsatzdefinition
-- revenue DECIMAL(8,2) GENERATED ALWAYS AS
--   (CAST(quantity * unit_price AS DECIMAL(8,2)))
```

Tabelle	Schlüssel und Aufgabe
dim_customer	customer_sk; SCD Type 2 auf loyalty_tier
dim_product	product_sk; Type 1 auf Produktattribute
dim_store	store_sk; Type 1 auf Filialattribute
dim_channel	channel_sk; in-store/Physical und mobile-app/Digital
dim_date	date_sk im Format YYYYMMDD; Kalender 2020 bis 2030
fact_order_items	order_item_sk; eine Zeile je Bestellposition

Kontrolle SHOW CREATE TABLE bestätigt customer_sk als Identity sowie effective_from DATE NOT NULL, effective_to DATE und is_current BOOLEAN. Die Faktentabelle enthält customer_sk, product_sk, store_sk, channel_sk, date_sk und die generierte Umsatzspalte.

Ergebnis Alle sechs Tabellen sind vorhanden; die geprüften Definitionen entsprechen den Vorgaben.

Abbildung 3 Sechs Warehouse Tabellen

	database	tableName	isTemporary
1	brewflow_warehou...	dim_channel	false
2	brewflow_warehou...	dim_customer	false
3	brewflow_warehou...	dim_date	false
4	brewflow_warehou...	dim_product	false
5	brewflow_warehou...	dim_store	false
6	brewflow_warehou...	fact_order_items	false

Schritte 5 und 6 Pipeline Queries und Dokumentation

Aufgabenstellung Drei gespeicherte SQL-Queries erstellen und die Tabellen sowie ihre Datenherkunft dokumentieren.

Schritt Staging-Tabellen mit leeren SELECT-Ergebnissen anlegen. Die drei Ladeskripte separat speichern und in Abhängigkeitsreihenfolge ausführen.

```
-- Muster zur Staging Einrichtung
CREATE TABLE IF NOT EXISTS workspace.brewflow_staging.stg_customers
USING DELTA
AS SELECT * FROM workspace.brewflow_oltp.customers WHERE 1 = 0;
-- Dasselbe Muster für stores products orders und order_items
SHOW TABLES IN workspace.brewflow_staging;
```

Aktion 03_copy_to_staging verwendet INSERT OVERWRITE. 04_load_dimensions nutzt MERGE und INSERT für die Kundenversionen sowie MERGE für die übrigen Dimensionen. 05_load_facts verwendet einen zeitlichen Kunden-Join und ergänzt nur neue Quellpositionen. Die vollständigen drei Ladeskripte stehen im Anhang.

Kontrolle Nach dem Erstlauf sind 10 Kundenversionen vorhanden, alle aktuell; Produkte 8, Filialen 5, Kanäle 2, Kalendertage 4018. Der Fakten-Load liefert 32 eindeutige Positionen, 0 fehlende Positionen und Umsatz 169,00.

Abbildung 4 Initialer Fakten Load

	staging_positions	fact_rows	unique_positions	missing_positions	total_revenue
1	32	32	32	0	169.00

Ergebnis Alle drei Queries wurden ausgeführt und gespeichert. Nach dem zweiten Durchlauf sind die geprüften Werte unverändert.

Abbildung 5 Kontrolle nach zweitem Durchlauf

	check_name	value
1	dim_customer_current	10.00
2	unique_order_items	32.00
3	total_revenue	169.00
4	dim_customer	10.00
5	dim_product	8.00
6	dim_store	5.00
7	dim_channel	2.00
8	dim_date	4018.00
9	fact_order_items	32.00

Genie erzeugte das Datenwörterbuch und die Spaltenherkunft für alle sechs Warehouse-Tabellen. Die Kommentarblöcke wurden geprüft, korrigiert und am Anfang der drei Queries gespeichert. Sie dokumentieren insbesondere feste Kanalwerte, Identity-Schlüssel und die Herkunft des Verkaufspreises aus order_items.

Schritt 7 Lakeflow Job und Zeitplan

Aufgabenstellung Die drei SQL-Queries automatisch nacheinander ausführen und einen täglichen Zeitplan einrichten.

Schritt Den Job brewflow_nightly_pipeline erstellen und alle Aufgaben mit dem Serverless Starter Warehouse (2XS) verbinden.

Aufgabe	SQL Query	Abhängig von
copy_to_staging	03_copy_to_staging	Keine
load_dimensions	04_load_dimensions	copy_to_staging
load_facts	05_load_facts	load_dimensions

Aktion Die Folgeaufgaben verwenden All succeeded. Der Trigger ist Scheduled und Active, mit täglicher Ausführung um 02:00 Uhr Europe/Berlin. Ein manueller Run-now-Lauf wurde gestartet.

Abbildung 6 Konfigurierter täglicher Zeitplan

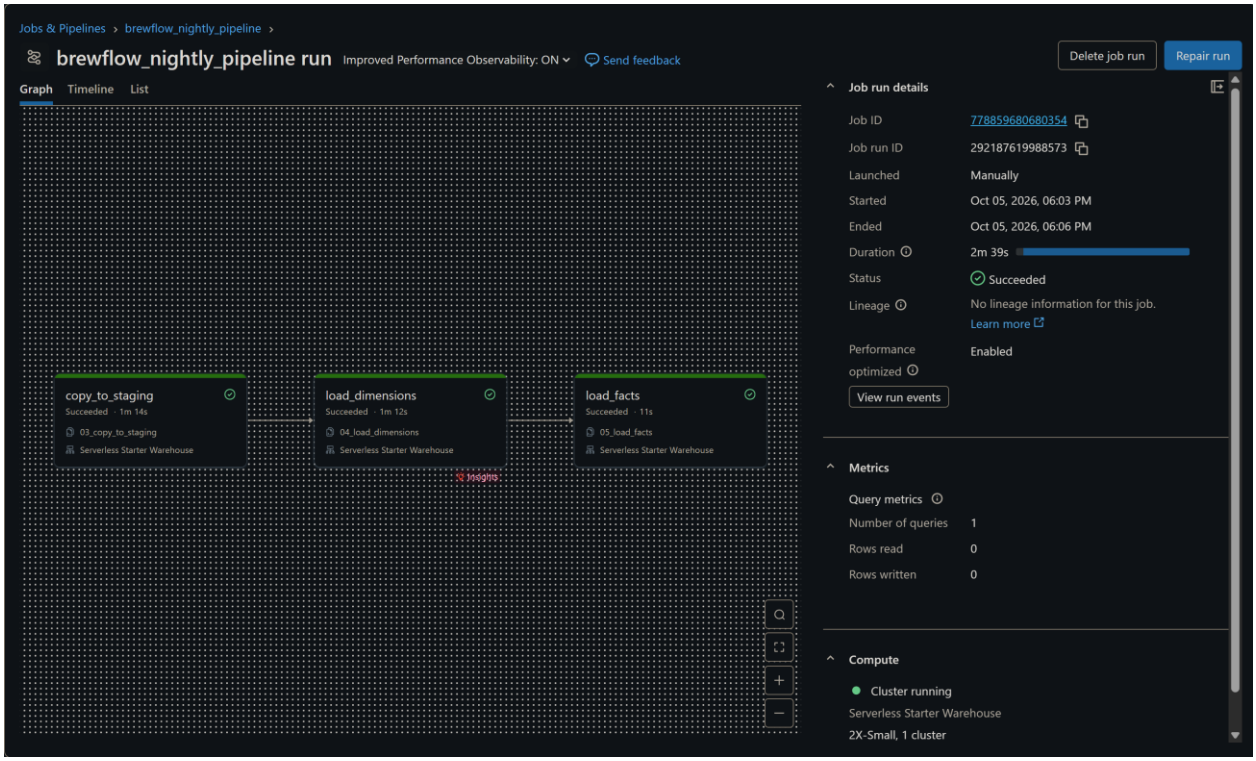


Kontrolle Der manuelle Lauf vom 5. Oktober 2026 zeigt Succeeded für alle drei Aufgaben in der richtigen Reihenfolge. Die Gesamtdauer beträgt 2 Minuten 39 Sekunden.

Ergebnis Job-Konfiguration und manueller Gesamtlauf sind durch die folgenden Abbildungen bestätigt. Ergebnisbericht und Zusammenfassung dokumentieren zusätzlich den erfolgreichen automatischen Nachtlauf vom 7. Oktober 2026.

Schritt 7 Nachweis des erfolgreichen Job Laufs

Abbildung 7 Manueller Job Lauf mit drei erfolgreichen Aufgaben



Aufgabenfolge: copy_to_staging → load_dimensions → load_facts. Die grünen Statusanzeigen bestätigen die erfolgreiche Ausführung aller drei SQL-Aufgaben.

Schritt 8 Szenario A Loyalty Upgrade

Aufgabenstellung Nach einem Loyalty-Upgrade die frühere Kundenversion erhalten und eine neue aktuelle Version anlegen.

Schritt Sofia Morales in der OLTP von Bronze auf Silver umstellen und den Job erneut starten.

```
-- Szenario A Änderung in der Quelle
UPDATE workspace.brewflow_oltp.customers
SET loyalty_tier = 'Silver'
WHERE customer_id = 1;

-- Nach erfolgreichem Job Lauf SCD Type 2 prüfen
SELECT customer_sk, customer_id, first_name, loyalty_tier,
       effective_from, effective_to, is_current
FROM workspace.brewflow_warehouse.dim_customer
WHERE customer_id = 1
ORDER BY effective_from;
```

Kontrolle Sofia besitzt zwei Versionen. Bronze mit customer_sk 1 gilt vom 15.03.2022 bis 04.10.2026 und ist nicht aktuell. Silver mit customer_sk 11 gilt ab 05.10.2026, ist aktuell und hat ein offenes Enddatum.

Schlüssel	Level	Gültig ab	Gültig bis	Aktuell
1	Bronze	15.03.2022	04.10.2026	false
11	Silver	05.10.2026	NULL	true

Ergebnis Bestanden. SCD Type 2 wurde anhand eines tatsächlichen Levelwechsels nachgewiesen.

Abbildung 8 Sofias historische und aktuelle Version



	customer_sk	customer_id	first_name	loyalty_tier	effective_from	effective_to	is_current
1	1	1	Sofia	Bronze	2022-03-15	2026-10-04	false
2	11	1	Sofia	Silver	2026-10-05	null	true

Die Erstversion beginnt bei joined_date. Dabei wird angenommen, dass der initial vorliegende Loyalty-Level ab dem Beitrittsdatum galt. Die OLTP enthält keine rückwirkende Historie. Folgeänderungen werden am Ladetag wirksam.

Schritt 8 Szenario B Neuer Kunde und Bestellung

Aufgabenstellung Einen neuen Kunden und seine Bestellung inkrementell in das Warehouse übernehmen.

Schritt Priya Sharma als Kundin 11, Bestellung 123 und Position 1033 ergänzen und anschließend den Job starten.

```
-- Szenario B Quellbestand erweitern
INSERT INTO workspace.brewflow_oltp.customers VALUES
(11, 'Priya', 'Sharma', 'priya.s@example.com',
 'Mumbai', 'Bronze', CURRENT_DATE());
INSERT INTO workspace.brewflow_oltp.orders VALUES
(123, 11, 3, CURRENT_DATE(), 'mobile-app', 4.20);
INSERT INTO workspace.brewflow_oltp.order_items VALUES
(1033, 123, 3, 1, 4.20);

-- Kontrolle nach dem Job Lauf
SELECT 'current_customers' AS check_name, COUNT(*) AS value
FROM workspace.brewflow_warehouse.dim_customer
WHERE is_current = TRUE
UNION ALL SELECT 'customer_versions', COUNT(*)
FROM workspace.brewflow_warehouse.dim_customer
UNION ALL SELECT 'fact_rows', COUNT(*)
FROM workspace.brewflow_warehouse.fact_order_items
UNION ALL SELECT 'total_revenue', SUM(revenue)
FROM workspace.brewflow_warehouse.fact_order_items;
```

Kontrolle 11 aktuelle Kunden, 12 Kundenversionen einschließlich Sofias Historie, 33 Fakten und Gesamtumsatz 173,20.

Ergebnis Bestanden. Neue Kunden und Positionen wurden ergänzt; die erwarteten bisherigen Zeilenanzahlen und der Umsatz bleiben konsistent.

Abbildung 9 Ergebnis der inkrementellen Ergänzung

	check_name	.00 value
1	current_customers	11.00
2	total_revenue	173.20
3	customer_versions	12.00
4	fact_rows	33.00

Schritt 8 Szenario C Verspätete Bestellung

Aufgabenstellung Eine heute geladene historische Bestellung dem zum Bestelldatum gültigen Loyalty-Level zuordnen.

Schritt Bestellung 124 mit Position 1034 vom 03.01.2024 für Sofia ergänzen und den Job erneut starten.

```
-- Szenario C Verspätete Quellposition
INSERT INTO workspace.brewflow_oltp.orders VALUES
(124, 1, 1, DATE '2024-01-03', 'in-store', 3.80);
INSERT INTO workspace.brewflow_oltp.order_items VALUES
(1034, 124, 2, 1, 3.80);

-- Historische Kundenzuordnung kontrollieren
SELECT f.order_item_id, f.order_id,
       dd.calendar_date AS order_date,
       f.customer_sk, dc.loyalty_tier, dc.is_current, f.revenue
FROM workspace.brewflow_warehouse.fact_order_items f
JOIN workspace.brewflow_warehouse.dim_customer dc
    ON f.customer_sk = dc.customer_sk
JOIN workspace.brewflow_warehouse.dim_date dd
    ON f.date_sk = dd.date_sk
WHERE f.order_item_id = 1034;
```

Kontrolle Position 1034 der Bestellung 124 trägt das Datum 03.01.2024 und customer_sk 1. Der zugehörige Loyalty-Level ist Bronze, is_current ist false und der Positionsumsatz beträgt 3,80.

Ergebnis Bestanden. Der Point-in-Time-Join wählt die historische Kundenversion anhand des Bestelldatums.

Abbildung 10 Historischer Bronze Bezug der verspäteten Position

	order_item_id	order_id	order_date	customer_sk	loyalty_tier	is_current	revenue
1	1034	124	2024-01-03	1	Bronze	false	3.80

Abschlusskontrolle und offene Arbeit

Prüfwert	Bestätigtes Ergebnis
current_customers	11
customer_versions	12
source_orders	24
source_positions	34
fact_rows	34
unique_positions	34
missing_positions	0
total_revenue	177,00

Kontrolle Die Abschlussabfrage vergleicht Kunden- und Versionsanzahlen, Quellbestellungen, Quellpositionen, Fakten und eindeutige Positions-IDs. Ein NOT-EXISTS-Abgleich ermittelt fehlende Positionen; SUM(revenue) liefert den Gesamtumsatz.

Ergebnis Alle erwarteten Werte sind bestätigt. Die Databricks-Pipeline und die drei Änderungsszenarien sind erfolgreich geprüft.

Abbildung 11 Finale Kontrolle nach allen Szenarien

	check_name	.00 value
1	current_customers	11.00
2	unique_positions	34.00
3	missing_positions	0.00
4	total_revenue	177.00
5	customer_versions	12.00
6	source_orders	24.00
7	source_positions	34.00
8	fact_rows	34.00

Die Power-BI-Verbindung und die Berichtstests sind abgeschlossen und in der folgenden Ergänzung dokumentiert. Die Präsentation ist erstellt. Das am 7. Oktober 2026 geprüfte lokale Abgabepaket umfasst 16 Dateien: sieben SQL-Dateien, die Job-Konfiguration, die Power-BI-Datei, vier Dokumentationsteile sowie Ergebnisbericht, Präsentation und Zusammenfassung als PDF. Die zuvor bestätigte Sicherung in Google Drive bezog sich auf einen früheren Dateistand. Vor der Abgabe ist sicherzustellen, dass dort die aktuellen Fassungen vorliegen und für die vorgesehenen Empfänger zugänglich sind.

Geltungsbereich: Der Fakten-MERGE ergänzt neue IDs und korrigiert keine bereits geladenen Positionen. Die SCD-Historisierung arbeitet auf Tagesebene; mehrere Levelwechsel am selben Tag wurden nicht getestet. Datenlöschungen und parallele Pipeline-Läufe sind nicht Bestandteil der vorliegenden Nachweise.

Anhang SQL Query 03 copy to staging

```
-- Script 03_copy_to_staging
-- Vollständige Kopie OLTP nach Staging
INSERT OVERWRITE workspace.brewflow_staging.stg_customers
SELECT * FROM workspace.brewflow_oltp.customers;
INSERT OVERWRITE workspace.brewflow_staging.stg_stores
SELECT * FROM workspace.brewflow_oltp.stores;
INSERT OVERWRITE workspace.brewflow_staging.stg_products
SELECT * FROM workspace.brewflow_oltp.products;
INSERT OVERWRITE workspace.brewflow_staging.stg_orders
SELECT * FROM workspace.brewflow_oltp.orders;
INSERT OVERWRITE workspace.brewflow_staging.stg_order_items
SELECT * FROM workspace.brewflow_oltp.order_items;
```

Anhang SQL Query 04 Kunden laden

Die folgenden beiden Seiten enthalten gemeinsam die vollständige Ausführungslogik von 04_load_dimensions.

```
-- Script 04_load_dimensions
-- Kunden Schritt 1 Geänderte Versionen beenden
MERGE INTO workspace.brewflow_warehouse.dim_customer AS target
USING workspace.brewflow_staging.stg_customers AS source
ON target.customer_id = source.customer_id AND target.is_current = TRUE
WHEN MATCHED AND NOT (target.loyalty_tier <=> source.loyalty_tier)
THEN UPDATE SET
    effective_to = DATE_SUB(CURRENT_DATE(), 1), is_current = FALSE;

-- Kunden Schritt 2 Neue oder geänderte Versionen ergänzen
INSERT INTO workspace.brewflow_warehouse.dim_customer (
    customer_id, first_name, last_name, email, city,
    loyalty_tier, effective_from, effective_to, is_current
)
SELECT src.customer_id, src.first_name, src.last_name,
       src.email, src.city, src.loyalty_tier,
       CASE WHEN EXISTS (
           SELECT 1 FROM workspace.brewflow_warehouse.dim_customer hist
           WHERE hist.customer_id = src.customer_id
       ) THEN CURRENT_DATE()
         ELSE COALESCE(src.joined_date, DATE '2020-01-01') END,
       CAST(NULL AS DATE), TRUE
FROM workspace.brewflow_staging.stg_customers src
WHERE NOT EXISTS (
    SELECT 1 FROM workspace.brewflow_warehouse.dim_customer dc
    WHERE dc.customer_id = src.customer_id AND dc.is_current = TRUE
);

-- Kunden Schritt 3 Beschreibende Attribute aktualisieren
MERGE INTO workspace.brewflow_warehouse.dim_customer AS target
USING workspace.brewflow_staging.stg_customers AS source
ON target.customer_id = source.customer_id AND target.is_current = TRUE
WHEN MATCHED THEN UPDATE SET
    first_name = source.first_name, last_name = source.last_name,
    email = source.email, city = source.city;
```

Anhang SQL Query 04 Weitere Dimensionen

```
-- Produkte Type 1 mit stabilen Schlüsseln
MERGE INTO workspace.brewflow_warehouse.dim_product AS target
USING workspace.brewflow_staging.stg_products AS source
ON target.product_id = source.product_id
WHEN MATCHED THEN UPDATE SET
    name = source.name, category = source.category,
    unit_price = source.unit_price
WHEN NOT MATCHED THEN INSERT (product_id, name, category, unit_price)
VALUES (source.product_id, source.name, source.category, source.unit_price);

-- Filialen Type 1 mit stabilen Schlüsseln
MERGE INTO workspace.brewflow_warehouse.dim_store AS target
USING workspace.brewflow_staging.stg_stores AS source
ON target.store_id = source.store_id
WHEN MATCHED THEN UPDATE SET
    name = source.name, city = source.city, country = source.country
WHEN NOT MATCHED THEN INSERT (store_id, name, city, country)
VALUES (source.store_id, source.name, source.city, source.country);

-- Verkaufskanäle Feste Referenzwerte
MERGE INTO workspace.brewflow_warehouse.dim_channel AS target
USING (
    SELECT 'in-store' AS channel, 'Physical' AS channel_group
    UNION ALL SELECT 'mobile-app', 'Digital'
) AS source
ON target.channel = source.channel
WHEN MATCHED THEN UPDATE SET channel_group = source.channel_group
WHEN NOT MATCHED THEN INSERT (channel, channel_group)
VALUES (source.channel, source.channel_group);

-- Kalender 2020 bis 2030
MERGE INTO workspace.brewflow_warehouse.dim_date AS target
USING (
    SELECT CAST(DATE_FORMAT(cal, 'yyyyMMdd') AS INT) AS date_sk,
           cal AS calendar_date, YEAR(cal) AS year,
           QUARTER(cal) AS quarter, MONTH(cal) AS month,
           DATE_FORMAT(cal, 'MMMM') AS month_name,
           DATE_FORMAT(cal, 'EEEE') AS day_of_week,
           DAYOFWEEK(cal) IN (1, 7) AS is_weekend
    FROM (
        SELECT EXPLODE(SEQUENCE(
            DATE '2020-01-01', DATE '2030-12-31', INTERVAL 1 DAY
        )) AS cal
    ) dates
) AS source
ON target.date_sk = source.date_sk
WHEN NOT MATCHED THEN INSERT (
    date_sk, calendar_date, year, quarter, month,
    month_name, day_of_week, is_weekend
) VALUES (
    source.date_sk, source.calendar_date, source.year,
    source.quarter, source.month, source.month_name,
    source.day_of_week, source.is_weekend
);
```

Anhang SQL Query 05 load facts

```
-- Script 05_load_facts
-- Neue Bestellpositionen mit zeitlicher Kundenzuordnung
MERGE INTO workspace.brewflow_warehouse.fact_order_items AS target
USING (
    SELECT oi.order_item_id, oi.order_id, dc.customer_sk,
           dp.product_sk, ds.store_sk, dch.channel_sk,
           dd.date_sk, oi.quantity, oi.unit_price
    FROM workspace.brewflow_staging.stg_order_items oi
    JOIN workspace.brewflow_staging.stg_orders o
        ON oi.order_id = o.order_id
    JOIN workspace.brewflow_warehouse.dim_customer dc
        ON o.customer_id = dc.customer_id
       AND o.order_date >= dc.effective_from
       AND (dc.effective_to IS NULL OR o.order_date <= dc.effective_to)
    JOIN workspace.brewflow_warehouse.dim_product dp
        ON oi.product_id = dp.product_id
    JOIN workspace.brewflow_warehouse.dim_store ds
        ON o.store_id = ds.store_id
    JOIN workspace.brewflow_warehouse.dim_channel dch
        ON o.channel = dch.channel
    JOIN workspace.brewflow_warehouse.dim_date dd
        ON o.order_date = dd.calendar_date
) AS source
ON target.order_item_id = source.order_item_id
WHEN NOT MATCHED THEN INSERT (
    order_item_id, order_id, customer_sk, product_sk, store_sk,
    channel_sk, date_sk, quantity, unit_price
) VALUES (
    source.order_item_id, source.order_id, source.customer_sk,
    source.product_sk, source.store_sk, source.channel_sk,
    source.date_sk, source.quantity, source.unit_price
);
```

Die Ausführungslogik entspricht den im Projekt verwendeten drei Queries. Die ausführlichen Datenwörterbuch- und Herkunftskommentare sind zusätzlich in den gespeicherten Databricks-Queries hinterlegt.

Power BI Verbindung und Datenmodell

Am 6. Oktober 2026 wurde Power BI Desktop mit dem Databricks SQL Warehouse verbunden. Die sechs Warehouse-Tabellen wurden geladen und unter ihren kurzen Tabellennamen übernommen. Die Datei BrewFlow_Final_Project.pbix wurde vom Nutzer gespeichert.

Die Verbindung verwendet den Databricks-Connector mit OAuth OIDC, den Katalog workspace und das Schema brewflow_warehouse. Als SQL Warehouse dient Serverless Starter Warehouse (2XS). Der verwendete Speichermodus wurde nicht separat dokumentiert.

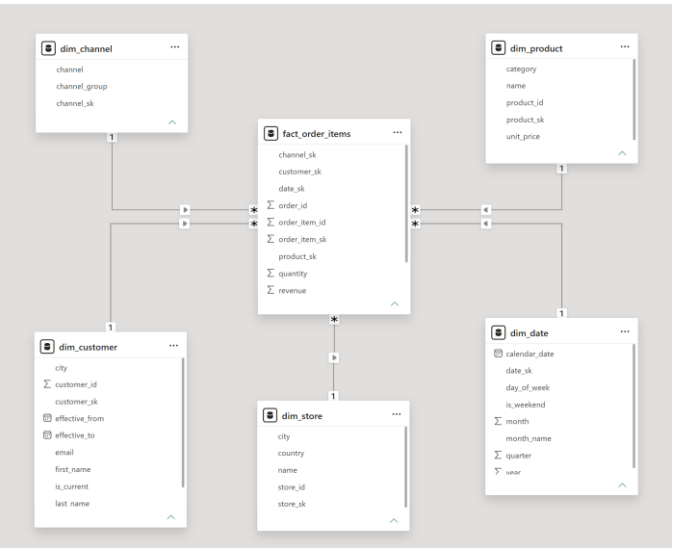
Beziehungen im Star Schema

Eine automatisch erzeugte Beziehung zwischen dim_customer.city und dim_store.city wurde entfernt. Die zunächst inaktive Filialbeziehung wurde aktiviert. Anschließend wurden alle fünf Schlüsselzuordnungen anhand der Beziehungseinstellungen kontrolliert.

Dimension	Schlüssel in Dimension und Fakt
dim_customer	customer_sk
dim_product	product_sk
dim_store	store_sk
dim_channel	channel_sk
dim_date	date_sk

Alle fünf Beziehungen sind aktiv und verbinden die Dimension auf der Eins-Seite mit fact_order_items auf der Viele-Seite. Die Kreuzfilterrichtung ist Einfach von der Dimension zur Faktentabelle. customer_sk erhält die Unterscheidung zwischen historischen Kundenversionen.

Abbildung 12 Bereinigtes Modell mit fünf aktiven Beziehungen



Power BI Kennzahlen und Dimensionsprüfung

Drei explizite Measures wurden in fact_order_items angelegt. Die ungefilterten Werte stimmen mit dem abschließenden Databricks-Bestand überein.

```
Gesamtumsatz = SUM(fact_order_items[revenue])
Bestellpositionen = COUNTROWS(fact_order_items)
Bestellungen = DISTINCTCOUNT(fact_order_items[order_id])
```

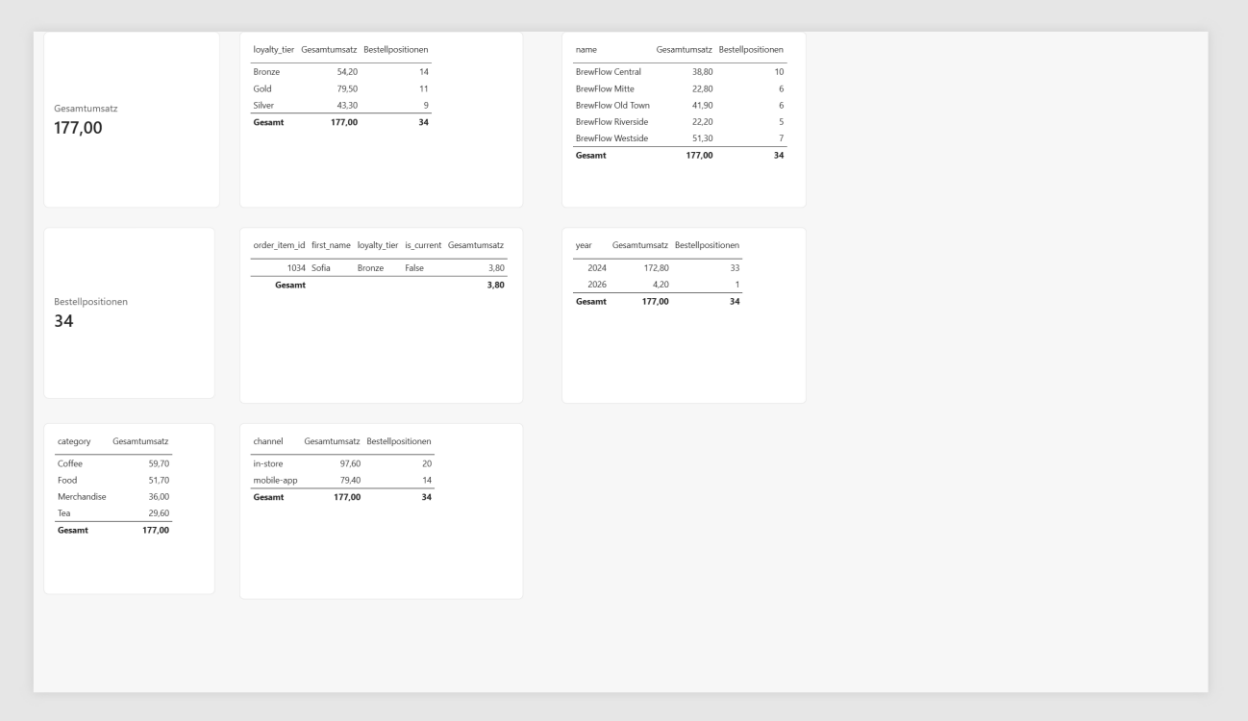
Kennzahl	Wert
Gesamtumsatz	177,00
Bestellpositionen	34
Bestellungen	24

Die Seite Datenprüfung enthält Tabellen nach Produktkategorie, Kundenstatus, Filiale, Kanal und Jahr. Jahreszahlen und Positions-IDs werden nicht summiert. In jeder vollständigen Aufteilung ergeben sich 177,00 Gesamtumsatz und 34 Positionen.

Verkaufskanal	Umsatz	Positionen
in-store	97,60	20
mobile-app	79,40	14
Gesamt	177,00	34
Jahr	**Umsatz**	**Positionen**
2024	172,80	33
2026	4,20	1
Gesamt	177,00	34

Power BI Nachweis der Dimensionsprüfungen

Abbildung 13 Prüfseite mit Dimensionsauswertungen und historischem Einzelnachweis



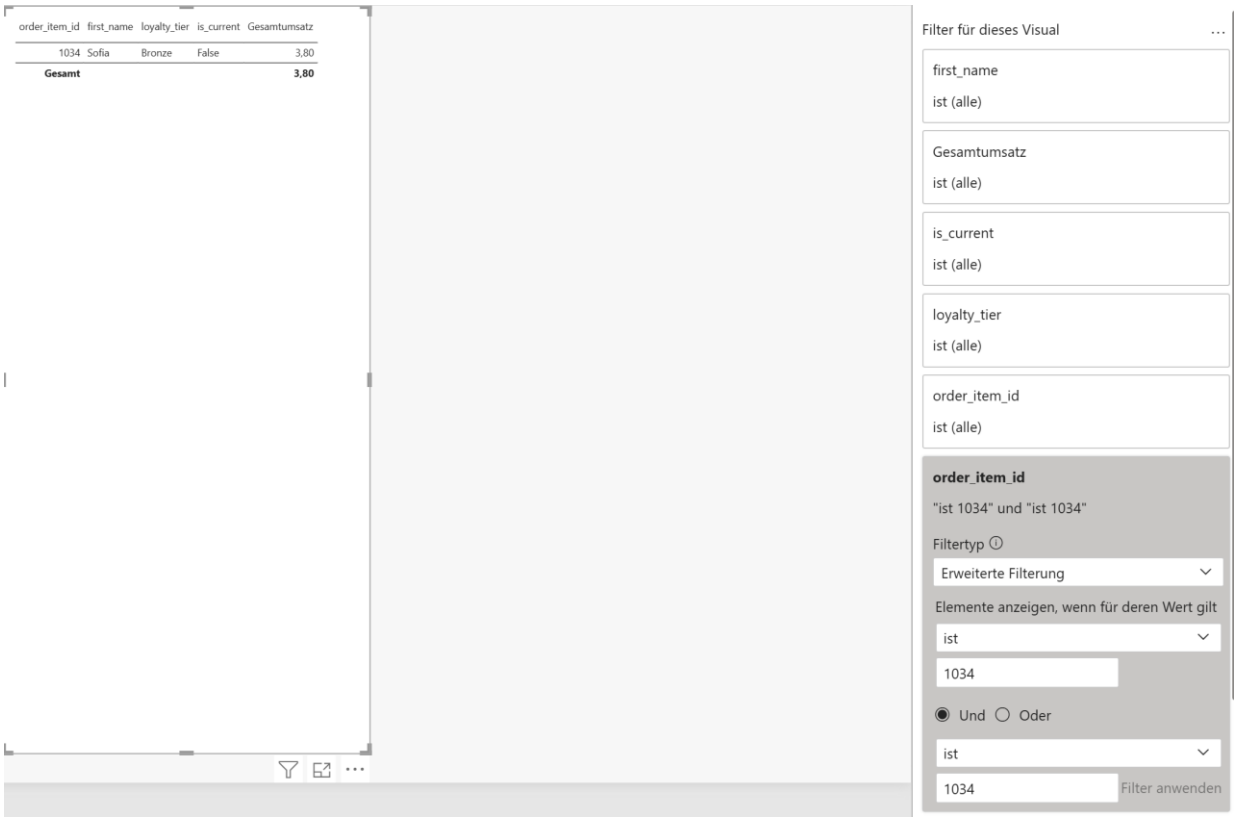
Power BI Nachweis der Kundenhistorie

Für die verspätet geladene Position 1034 wurde eine Tabelle mit order_item_id, first_name, loyalty_tier, is_current und Gesamtumsatz erstellt. Ein Filter auf Visualebene begrenzt den Nachweis auf diese Position.

Position	Kundin	Status	Aktuell	Umsatz
1034	Sofia	Bronze	False	3,80

Der Bericht zeigt Sofias frühere Bronze-Version, obwohl ihre aktuelle Version Silver ist. Damit ist die in Databricks geprüfte Zuordnung zur historischen Kundenversion auch im Power-BI-Bericht sichtbar. is_current beschreibt die Kundenversion und nicht die Aktualität der Bestellung.

Abbildung 14 Historische Position 1034 in Power BI

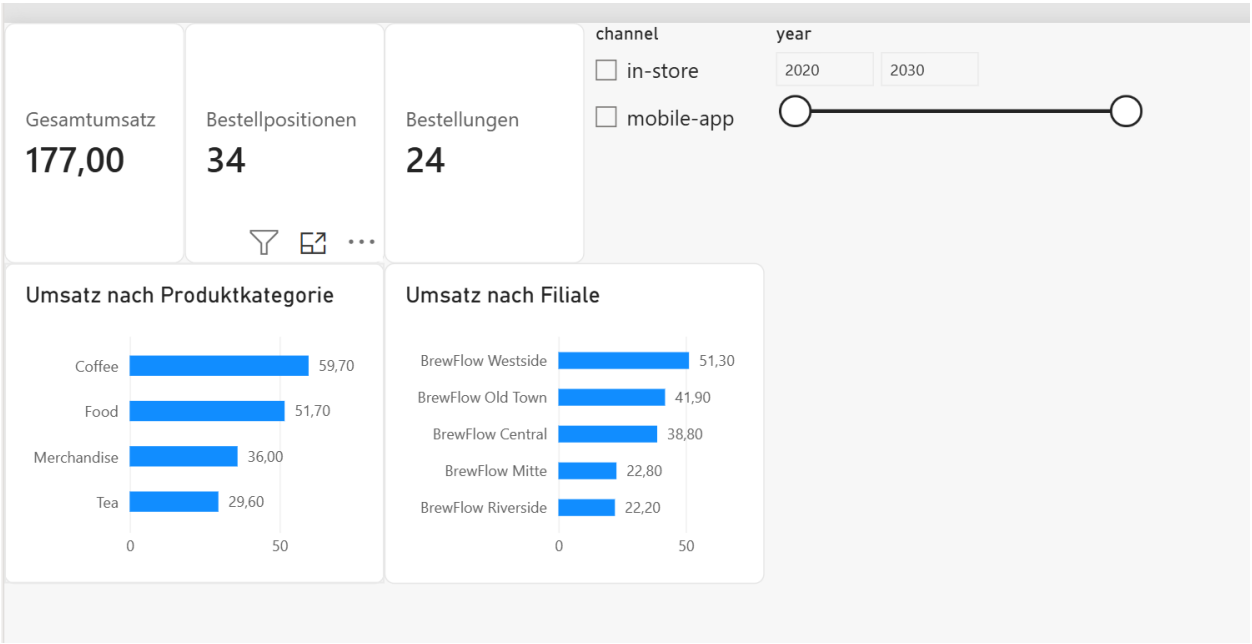


Das Filterbild enthält dieselbe Bedingung zweimal mit UND. Beide Bedingungen wählen Position 1034 aus; diese redundante Konfiguration ändert das Ergebnis nicht.

Power BI Übersicht und interaktive Filter

Die Seite BrewFlow Übersicht enthält drei Kennzahlenkarten, drei nach Umsatz absteigend sortierte Balkendiagramme und ein Liniendiagramm für den Umsatz nach Kanal im Zeitverlauf. Die Balkendiagramme zeigen Produktkategorien, Filialen und Städte mit Werten auf zwei Dezimalstellen. Datenschnitte filtern Verkaufskanal und Jahr. Automatische Untertitel und Gitternetzlinien der Berichtsfläche wurden entfernt.

Abbildung 15 Übersicht vor Ergänzung der Stadt und Zeitauswertung

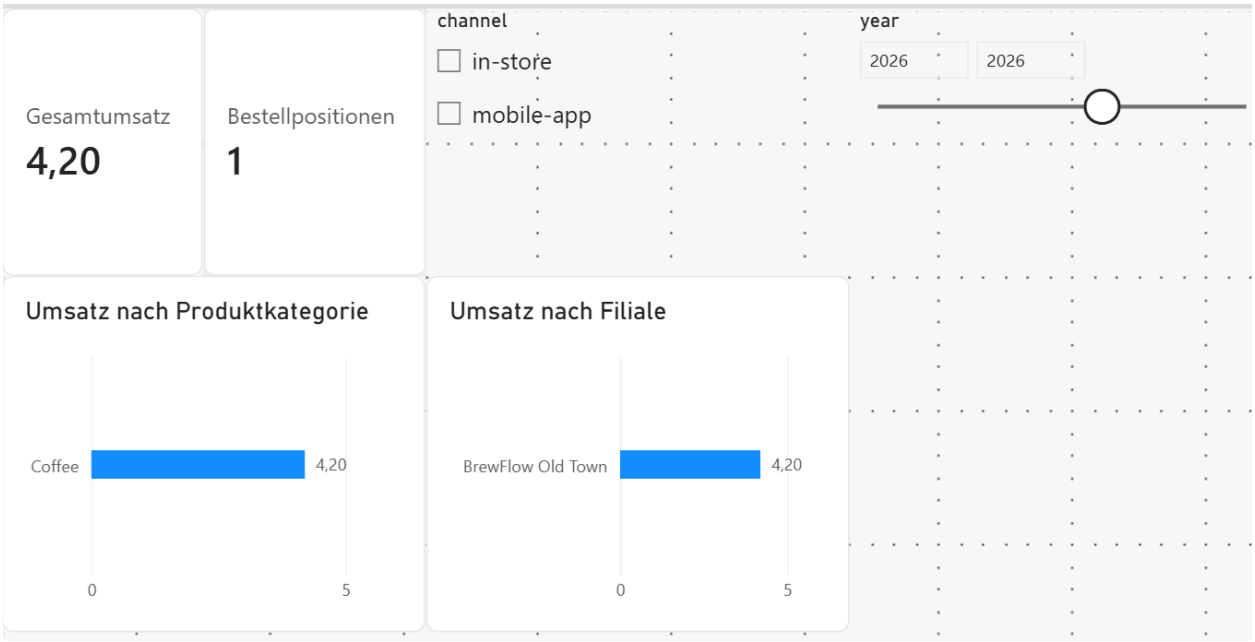


Filtertest	Umsatz	Positionen
mobile-app	79,40	14
in-store	97,60	20
Nur 2026	4,20	1
Alle Kanäle und 2020 bis 2030	177,00	34

Die Auswahl eines Kanals änderte beide Kennzahlen und beide Diagramme. Der Test für ausschließlich 2026 zeigte Coffee und BrewFlow Old Town mit 4,20 Umsatz und einer Position. Nach dem Test wurden die Filter zurückgesetzt und die Datei gespeichert.

Power BI Nachweis des Jahresfilters

Abbildung 16 Bericht für ausschließlich 2026



Der geforderte Power-BI-Umfang ist einschließlich der Stadt- und Kanalauswertung im Zeitverlauf umgesetzt. Eine Veröffentlichung im Power BI Service oder eine automatische Aktualisierung wurde nicht eingerichtet oder geprüft. Eine frühere Sicherung in Google Drive wurde vom Nutzer bestätigt. Die Vollständigkeit der aktuellen Fassungen und die Zugriffsrechte für die vorgesehenen Empfänger müssen direkt im Abgabeordner kontrolliert werden; aus den lokalen Projektdateien allein lässt sich diese Freigabe nicht bestätigen.

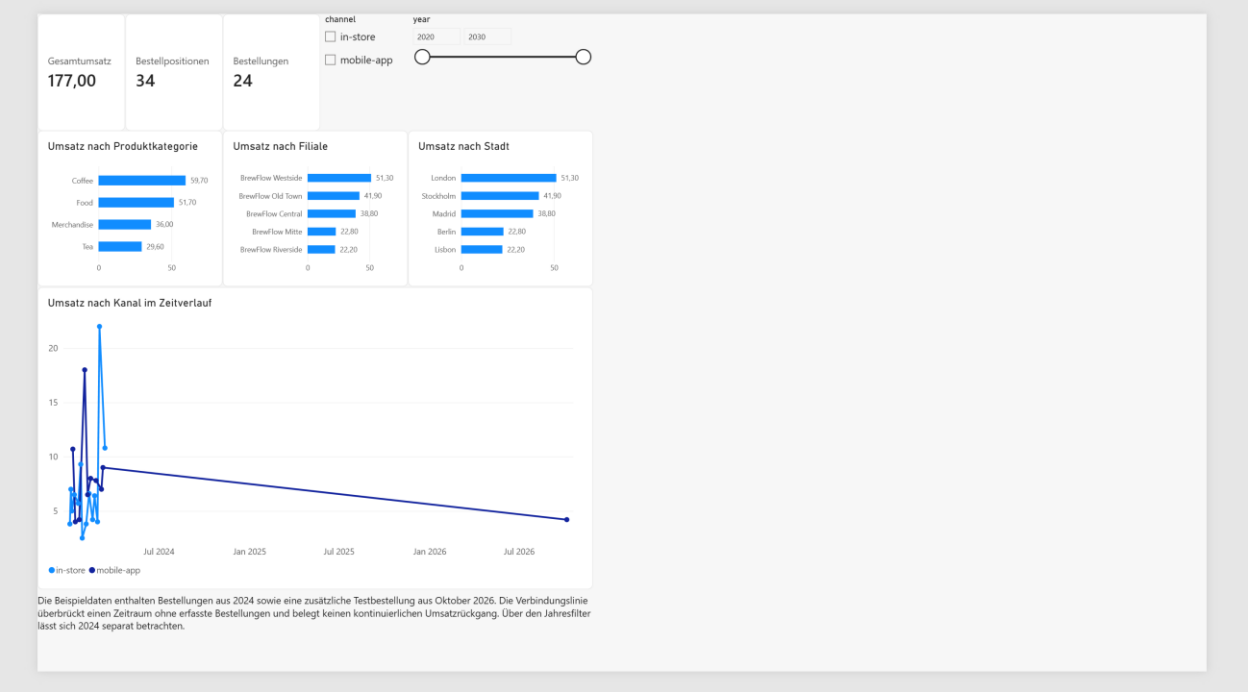
Power BI Stadt und Kanal im Zeitverlauf

Die Standortauswertung verwendet city aus dim_store und das Measure Gesamtumsatz. London erzielt 51,30, Stockholm 41,90, Madrid 38,80, Berlin 22,80 und Lisbon 22,20. Die Summe beträgt 177,00. Im vorliegenden Datensatz hat jede Filiale einen anderen Stadtwert; deshalb entsprechen die Stadtsummen den jeweiligen Filialsummen.

Das Zeitdiagramm verwendet calendar_date aus dim_date auf der X-Achse, Gesamtumsatz auf der Y-Achse und channel aus dim_channel als Legende. Die automatische Datumshierarchie wurde durch das konkrete Datum ersetzt. Beide Datenreihen zeigen Marker und lineare Verbindungslinien.

Die Bestellungen liegen überwiegend Anfang 2024. Die zusätzliche mobile-app-Testbestellung über 4,20 liegt im Oktober 2026. Die lange Verbindung überbrückt einen Zeitraum ohne erfasste Bestellungen und belegt keinen kontinuierlichen Umsatzrückgang. Dieser Hinweis steht direkt unter der Grafik; der Jahresfilter ermöglicht eine separate Betrachtung von 2024.

Abbildung 17 Vollständige Übersicht mit Stadt Zeitauswertung und Erklärung



Die ungefilterte Übersicht zeigt weiterhin 177,00 Umsatz, 24 Bestellungen und 34 Positionen. Der ergänzte Bericht ist als Power-BI-Datei Teil des vom Nutzer erneut hochgeladenen Abgabebestands.